CS339: Abstractions and Paradigms for Programming

Metacircular Evaluator (Cont.)

Manas Thakur
CSE, IIT Bombay

Autumn 2025

Expanding features:
Variable number of arguments

Lexical vs Dynamic Scoping

- Do we need a keyword or a symbol?
  - ?, :, if
- How much would we be constraining the programmers as a result of our choice?
  - Underscores
- Can we introduce it unambiguously?
  - Problems with ambiguity?
    - Hyphens, if-else chaining, …

```
if (c1) S1
else if (c2) S2
else S3
```

Syntax and semantics for varargs

```
(define foo
  (lambda (x . y)
    (+ x (sum-list y))))

(define (sum-list
  (lambda (l)
    (foldr + l 0)))

> (foo 2 5 7 9)
```

What happens without the '.'?

- x takes the first argument
- y takes the rest of the arguments
  - y has to be a list.

Changes in the interpreter

- Standard way of **pairing up** actual argument values with formal parameter variables:

```
(define pair-up
  (lambda (vars vals)
    (cond
      ((eq? vars '())
       (cond ((eq? vals '()) '())
             (else (error "TMA"))))
      ((eq? vals '()) (error "TFA"))
      (else
       (cons (cons (car vars)
                   (car vals))
             (pair-up (cdr vars)
                      (cdr vals)))))))
```

Changes in the interpreter (Cont.)

➤ Pairing up argument values with parameter variables to support varargs:

```
(define pair-up
  (lambda (vars vals)
    (cond
      ((eq? vars '())
       (cond ((eq? vals '()) '())
             (else (error "TMA"))))
      ((symbol? vars)
       (cons (cons vars vals) '()))
      ((eq? vals '()) (error "TFA"))
      (else
       (cons (cons (car vars)
                   (car vals))
             (pair-up (cdr vars)
                      (cdr vals)))))))
```

Exploring design choices: Lexical vs Dynamic Scoping

Lexical vs Dynamic Scoping

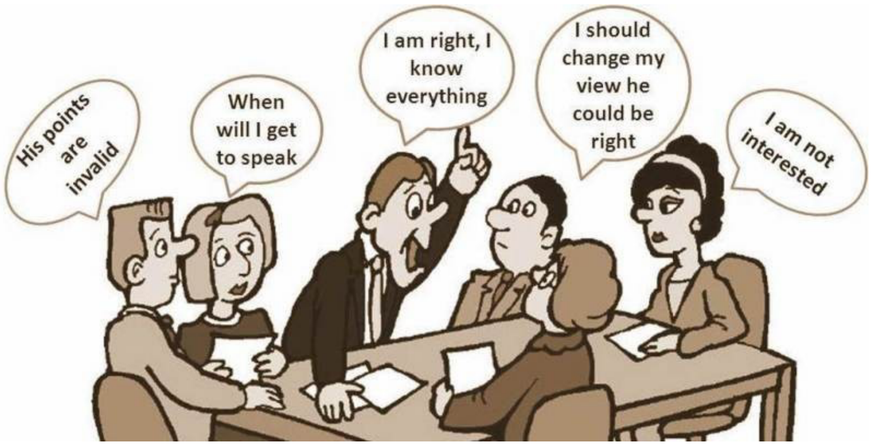
- Lexical scoping: Free variables are bound in the environment in which the procedure was defined (closure).
- Dynamic scoping: Free variables are bound the most recently assigned value during program execution.

```
x <- 1
f <- function(a) x + a
g <- function() {
     x <- 2
     f(0)
}
g()
```

- Learn more about various scoping considerations (and confusions):

Discussion

- Which one — lexical or dynamic scoping — is more natural?
- Which one should be easier to implement?
- Original Lisp had dynamic scoping!
- Then why switch to lexical scoping in Scheme?



Flashback

- Recall the general sum procedure from the days we learnt about higher order functions:

```
(define (sum term a next b)
  (if (> a b)
      0
      (+ (term a)
         (sum term (next a) next b))))
```

- Usage:

```
(define (sum-cubes a b)
  (define (cube x) (expt x 3))
  (sum cube a inc b))
```

- One more (sum of nth powers):

```
(define (sum-powers a b n)
  (define (nth-power x) (expt x n))
  (sum nth-power a inc b))
```

Notice the binding

Why not Dynamic Scoping?

Problem with Dynamic Scoping

- Say the programmer renamed b in the procedure sum to n:

```
(define (sum term a next n)
  (if (> a n)
      0
      (+ (term a)
         (sum term (next a) next n))))

(define (sum-powers a b n)
  (define (nth-power x) (expt x n))
  (sum nth-power a inc b))
```

- Oops! Now we have a problem.

n got bound to b

- We broke a core abstraction barrier (that procedures could rename parameters)

Then why Dynamic Scoping?

Classwork

- Write an analogous procedure to multiply the nth powers.

```
(define (product-powers a b n)
  (define (nth-power x) (expt x n))
  (product nth-power a inc b))
```

- Can we abstract out nth-power?
  - Lexical scoping: No.
  - Dynamic scoping: Yes!

```
(define (sum-powers a b n)
  (define (nth-power x) (expt x n))
  (sum nth-power a inc b))
```

```
(define (sum-powers a b n)
  (sum nth-power a inc b))

(define (product-powers a b n)
  (product nth-power a inc b))

(define (nth-power x)
  (expt x n))
```

Now comes fun!

Make Scheme Dynamically Scoped

- Remarkably simple: Just extend the correct environment!

```
(define eval
  (lambda (exp env)
    (cond ((number? exp) exp)
          ...
          ((pair? exp) (apply (eval (car exp) env) (evallist (cdr exp) env)))
          (else (error "unknown expression type")))))

(define (apply
  (lambda (proc args)
    (cond ((primitive-op? proc) (apply-prim-op proc args))
          ((compound-op? proc) (eval (caddr proc) (extend-environ (cadr proc) args (cadddr proc)))))))
```

```
(define eval
  (lambda (exp env)
    (cond ((number? exp) exp)
          ...
          ((pair? exp) (apply (eval (car exp) env) (evallist (cdr exp) env) env))
          (else (error "unknown expression type")))))

(define (apply
  (lambda (proc args env)
    (cond ((primitive-op? proc) (apply-prim-op proc args))
          ((compound-op? proc) (eval (caddr proc) (extend-environ (cadr proc) args env)))))))
```

In fact, we won't need to bind the environment while creating procedure objects!

Scoping in Action

Insight of the Week (Semester!)

- How do we abstract out nth-power with lexical scoping?

```
(define make-exp
  (lambda (n)
    (lambda (x) (expt x n))))
```

By learning
the APP course :-)

```
(define (sum-powers a b n)
  (sum (make-exp n) a inc b))
```

```
(define (product-powers a b n)
  (product (make-exp n) a inc b))
```

- What's the powerful feature that allowed you to solve this problem?
  - Ability to return functions as values.
  - In general, granting **first-class citizenship to functions**!